Please solve all these questions given in the attached PDF with good understadable explanations

I'll solve all the Dynamic Programming problems from your PDF with detailed, understandable explanations.[1]

Knapsack DP Problems

Problem 1: Dice Sum Ways

Problem: Count the number of ways to construct sum n by throwing a dice (outcomes 1-6) one or more times.[1]

State: $\mathrm{dp}[i]$ = number of ways to construct sum i.[1]

Base Case: $\mathrm{dp} = 1$ (one way to make 0: throw no dice).[1]

Transition: For each sum i, we can reach it by throwing any dice value (1-6) from a previous sum:
$\mathrm{dp}[i] = \sum_{j=1}^{6} \mathrm{dp}[i-j] \ (\text{where } i-j \geq 0)$

Target: Return $\mathrm{dp}[n]$.[1]

Algorithm:

```
def countDiceWays(n):
    dp = [^0] * (n + 1)
    dp[^0] = 1

    for i in range(1, n + 1):
        for dice in range(1, 7):
            if i - dice >= 0:
                dp[i] += dp[i - dice]

    return dp[n]
```

Correctness: Each state $\mathrm{dp}[i]$ considers all possible last dice throws. Since dice outcomes are independent, summing all ways from previous states gives total ways.[1]

Complexity:

- Time: $O(6n) = O(n)$

- Space: $O(n)$ [1]

Example: For $n = 3$: Ways = {1+1+1, 1+2, 2+1, 3} = 4 ways.[1]

Problem 2: Coin Combinations (Order Matters)

Problem: Count distinct ways to produce sum x using coins, where **order matters** (2+3 and 3+2 are different).[1]

State: $\mathrm{dp}[i]$ = number of ways to make sum i.[1]

Base Case: $\mathrm{dp} = 1$.[1]

Transition: For each sum i, try using each coin c_j:
$\mathrm{dp}[i] = \sum_{j=1}^{n} \mathrm{dp}[i - c_j] \ (\text{where } i - c_j \geq 0)$

Target: Return $\mathrm{dp}[x]$.[1]

Algorithm:

```
def coinCombinations(coins, x):
    dp = [^0] * (x + 1)
    dp[^0] = 1

    for i in range(1, x + 1):
        for coin in coins:
            if i - coin >= 0:
                dp[i] += dp[i - coin]

    return dp[x]
```

Why order matters: We iterate over sums first, then coins. For each sum, we consider adding any coin, which creates different orderings.[1]

Complexity:

- Time: $O(nx)$
- Space: $O(x)$ [1]

Example: Coins = {2, 3, 5}, sum = 9: Answer = 8 ways (as given in problem).[1]

Problem 3 & 4: Ordered Combinations (Order Doesn't Matter)

Problem: Count distinct **ordered** ways (combinations, not permutations) where order doesn't matter.[1]

State: $\mathrm{dp}[i]$ = number of ways to make sum i.[1]

Base Case: $\mathrm{dp} = 1$.[1]

Transition: Iterate over coins first, then sums:
$\mathrm{dp}[i] = \mathrm{dp}[i] + \mathrm{dp}[i - c_j]$

This ensures we only count combinations, not permutations.[1]

Algorithm:

```
def orderedCombinations(coins, x):
    dp = [^0] * (x + 1)
    dp[^0] = 1

    for coin in coins:  # Iterate coins first!
        for i in range(coin, x + 1):
            dp[i] += dp[i - coin]

    return dp[x]
```

Why this works: By processing coins in order, we ensure that for each coin, we only add it after considering all smaller coins. This prevents counting the same combination multiple times.[1]

Complexity:

- Time: $O(nx)$
- Space: $O(x)$[1]

Example: Coins = {2, 3, 5}, sum = 9: Answer = 3 ways: {2+2+5, 3+3+3, 2+2+2+3}.[1]

Problem 5: Minimize Coins

Problem: Find minimum number of coins needed to make sum x.[1]

State: $\mathrm{dp}[i]$ = minimum number of coins to make sum i.[1]

Base Case:

- $\mathrm{dp} = 0$
- $\mathrm{dp}[i] = \infty$ for $i > 0$[1]

Transition: For each sum i, try each coin c_j:
$\mathrm{dp}[i] = \min_j(\mathrm{dp}[i - c_j] + 1)\ (\text{where } i - c_j \geq 0)$

Target: Return $\mathrm{dp}[x]$ if finite, else "impossible".[1]

Algorithm:

```
def minimizeCombinations(coins, x):
    dp = [float('inf')] * (x + 1)
    dp[^0] = 0

    for i in range(1, x + 1):
        for coin in coins:
            if i - coin >= 0:
```

```
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[x] if dp[x] != float('inf') else -1
```

Correctness: At each sum, we consider all possible last coins and take the minimum.[1]

Complexity:

- Time: $O(nx)$
- Space: $O(x)$[1]

Example: Coins = {1, 5, 7}, sum = 11: Answer = 3 (5+5+1).[1]

Problem 6: 0-1 Knapsack

Problem: Given N items with weights w_i and values v_i, and knapsack capacity W, maximize total value.[1]

State: $\mathrm{dp}[i][w]$ = maximum value using first i items with capacity w.[1]

Base Case: $\mathrm{dp}[w] = 0$ for all w (no items, no value).[1]

Transition: For each item i and capacity w:

$$\mathrm{dp}[i][w] = \max \begin{cases} \mathrm{dp}[i-1][w] & (\text{don't take item } i) \\ \mathrm{dp}[i-1][w-w_i] + v_i & (\text{take item } i, \text{if } w \geq w_i) \end{cases}$$

Target: Return $\mathrm{dp}[N][W]$.[1]

Algorithm:

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[^0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i-1][w]  # Don't take
            if w >= weights[i-1]:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])

    return dp[n][W]
```

Correctness: Each state considers the optimal choice of including or excluding the current item.[1]

Complexity:

- Time: $O(NW)$
- Space: $O(NW)$ (can be optimized to $O(W)$ using 1D array)[1]

Problem 7: All Money Sums

Problem: Find all possible money sums you can create using n coins.[1]

State: $\text{possible}[s]$ = True if sum s is achievable.[1]

Base Case: $\text{possible} = \text{True}$.[1]

Transition: For each coin c, mark all sums that can be reached by adding c to existing achievable sums:

```
for existing_sum in achievable_sums:
    possible[existing_sum + c] = True
```

Target: Return all indices where $\text{possible}[s] = \text{True}$.[1]

Algorithm:

```
def moneySums(coins):
    max_sum = sum(coins)
    possible = [False] * (max_sum + 1)
    possible[^0] = True

    for coin in coins:
        # Iterate backwards to avoid using same coin twice
        for s in range(max_sum, coin - 1, -1):
            if possible[s - coin]:
                possible[s] = True

    return [s for s in range(1, max_sum + 1) if possible[s]]
```

Why iterate backwards: Prevents using the same coin multiple times in one iteration.[1]

Complexity:

- Time: $O(n \cdot \sum \text{coins})$
- Space: $O(\sum \text{coins})$[1]

Miscellaneous DP Problems

Problem 8: Matrix Chain Multiplication Example

Problem: Find optimal parenthesization for matrices with dimensions {5, 10, 3, 12, 5, 50, 6}.[1]

State: $\text{dp}[i][j]$ = minimum number of scalar multiplications to compute $A_i \times A_{i+1} \times \ldots \times A_j$.[1]

Dimensions:

- Matrix 1: 5×10
- Matrix 2: 10×3
- Matrix 3: 3×12
- Matrix 4: 12×5
- Matrix 5: 5×50
- Matrix 6: 50×6 [1]

Recurrence:
$\text{dp}[i][j] = \min_{i \le k < j} \{\text{dp}[i][k] + \text{dp}[k+1][j] + d_{i-1} \cdot d_k \cdot d_j\}$

where d is the dimension array.[1]

Solution Steps:

1. Base case: $\text{dp}[i][i] = 0$ (single matrix, no multiplication)
2. Fill for chains of length 2, 3, ..., 6
3. Track split points in $\text{split}[i][j]$

Computation:

```
Length 2:
dp[^1][^2] = 5×10×3 = 150
dp[^2][^3] = 10×3×12 = 360
dp[^3][^4] = 3×12×5 = 180
dp[^4][^5] = 12×5×50 = 3000
dp[^5][^6] = 5×50×6 = 1500

Length 3:
dp[^1][^3] = min(dp[^1][^1]+dp[^2][^3]+5×10×12=600,
                 dp[^1][^2]+dp[^3][^3]+5×3×12=330) = 330

Continue for all lengths...
```

Optimal Parenthesization: After computing the full table, backtrack using split points to construct: $((A_1(A_2A_3))((A_4A_5)A_6))$.[1]

Complexity:

- Time: $O(n^3)$
- Space: $O(n^2)$ [1]

Problem 9: Counting Parentheses

Proof by Induction: Show that a full parenthesization of an n-element expression has exactly $n-1$ pairs of parentheses.[1]

Base Case: For $n = 1$, we have a single element with no parentheses: 0 = 1-1 ✓ [1]

Inductive Step: Assume true for all expressions with fewer than n elements. For an n-element expression, the full parenthesization splits it into two parts at some position: $(E_1)(E_2)$ where E_1 has k elements and E_2 has $n-k$ elements.[1]

By the inductive hypothesis:

- E_1 has $k-1$ pairs of parentheses
- E_2 has $(n-k)-1$ pairs of parentheses

Total pairs = $(k-1)+(n-k-1)+1 = n-1$ (the +1 is for the outer pair).[1]

Problem 10: Print Optimal Parenthesization

Algorithm:

```
def printOptimalParens(split, i, j, name='A'):
    if i == j:
        print(f"{name}{i}", end='')
    else:
        print("(", end='')
        printOptimalParens(split, i, split[i][j], name)
        printOptimalParens(split, split[i][j] + 1, j, name)
        print(")", end='')
```

Explanation: The $\text{split}[i][j]$ array stores the optimal split point for multiplying matrices i through j. Recursively print left and right parts.[1]

Complexity: $O(n)$ since we visit each matrix once.[1]

Problem 11: MCM Subproblem Graph

Vertices: Each subproblem $\text{dp}[i][j]$ where $1 \leq i \leq j \leq n$ is a vertex.[1]

Number of vertices: $\binom{n}{2} + n = \frac{n(n+1)}{2} = O(n^2)$[1]

Edges: There's an edge from $\text{dp}[i][j]$ to $\text{dp}[i][k]$ and $\text{dp}[k+1][j]$ for all valid split points k where $i \leq k < j$.[1]

Number of edges: For each vertex (i,j) where $i<j$, there are $j-i$ edges (one for each split point). Total edges:
$\sum_{i=1}^{n}\sum_{j=i+1}^{n}(j-i) = \sum_{len=1}^{n-1} len \cdot (n-len) = O(n^3)$

Structure: The graph is a DAG with dependencies flowing from smaller subproblems to larger ones.[1]

Problem 12: Fibonacci in O(n) Time

State: $\mathrm{dp}[i]$ = i-th Fibonacci number.[1]

Base Cases:

- $\mathrm{dp} = 0$ (or 1, depending on convention)
- $\mathrm{dp} = 1$[2] [1]

Transition: $\mathrm{dp}[i] = \mathrm{dp}[i-1] + \mathrm{dp}[i-2]$.[1]

Algorithm:

```
def fibonacci(n):
    if n <= 1:
        return n

    dp = [^0] * (n + 1)
    dp[^0], dp[^1] = 0, 1

    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]
```

Space-Optimized Version:

```
def fibonacci(n):
    if n <= 1:
        return n

    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current

    return prev1
```

Subproblem Graph:

- **Vertices:** $n + 1$ vertices (one for each Fibonacci number from 0 to n)
- **Edges:** $2(n-1)$ edges (each vertex i has edges to $i-1$ and $i-2$ for $i \geq 2$)
- Structure is a simple path with backward dependencies[1]

Complexity:

- Time: $O(n)$
- Space: $O(1)$ with optimization[1]

Problem 13: Maximize Matrix Multiplications

Question: Does the problem of maximizing (instead of minimizing) scalar multiplications exhibit optimal substructure?[1]

Answer: Yes, it does exhibit optimal substructure.[1]

Proof: Similar to the minimization version, to maximize multiplications for chain $A_i \cdots A_j$, we need to:

1. Choose a split point k
2. Maximize multiplications for $A_i \cdots A_k$
3. Maximize multiplications for $A_{k+1} \cdots A_j$
4. Add the cost of multiplying the two resulting matrices[1]

The optimal solution to the larger problem contains optimal solutions to subproblems. The recurrence becomes:
$\text{dp}[i][j] = \max_{i \leq k < j} \{\text{dp}[i][k] + \text{dp}[k+1][j] + d_{i-1} \cdot d_k \cdot d_j\}$

This is structurally identical to the minimization problem, just with max instead of min.[1]

Problem 14: Print LIS (Longest Increasing Subsequence)

Algorithm:

```
def printLIS(arr):
    n = len(arr)
    dp = [^1] * n  # Length of LIS ending at i
    parent = [-1] * n  # Previous index in LIS

    # Compute LIS lengths and track parents
    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
                parent[i] = j

    # Find index with maximum LIS length
    max_length = max(dp)
    max_idx = dp.index(max_length)

    # Reconstruct LIS by backtracking
    lis = []
    idx = max_idx
    while idx != -1:
        lis.append(arr[idx])
        idx = parent[idx]

    return lis[::-1]  # Reverse to get correct order
```

Explanation:

- $\text{dp}[i]$ stores length of LIS ending at index i
- $\text{parent}[i]$ stores the previous index in the optimal LIS
- Backtrack from the index with maximum length to reconstruct the sequence[1]

Complexity:

- Time: $O(n^2)$ for computation + $O(n)$ for reconstruction = $O(n^2)$
- Space: $O(n)$[1]

Problem 15: LCS (Longest Common Subsequence) Length

Problem: Given two DNA sequences S_1 and S_2, find the length of their longest common subsequence.[1]

State: $\text{dp}[i][j]$ = length of LCS of $S_1[1..i]$ and $S_2[1..j]$.[1]

Base Cases:

- $\text{dp}[j] = 0$ for all j
- $\text{dp}[i] = 0$ for all i[1]

Transition:

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] + 1 & \text{if } S_1[i] = S_2[j] \\ \max(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{otherwise} \end{cases}$$

Algorithm:

```
def LCS_Length(S1, S2):
    m, n = len(S1), len(S2)
    dp = [[^0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if S1[i-1] == S2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

Proof of Correctness:
Let X_i denote $S_1[1..i]$ and Y_j denote $S_2[1..j]$. We prove by induction on $i + j$:

Base: When $i = 0$ or $j = 0$, LCS is empty, length 0. ✓

Inductive Step: Assume correctness for all pairs with sum < $i + j$. For (i, j):

- If $S_1[i] = S_2[j]$: Any LCS must include this character. The remaining LCS is of X_{i-1} and Y_{j-1}, which by IH is $\text{dp}[i-1][j-1]$. So $\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$ ✓

- If $S_1[i] \neq S_2[j]$: LCS either doesn't include $S_1[i]$ (so it's LCS of X_{i-1}, Y_j) or doesn't include $S_2[j]$ (LCS of X_i, Y_{j-1}). Take maximum ✓ [1]

Complexity:

- Time: $O(mn)$
- Space: $O(mn)$ (can be optimized to $O(\min(m, n))$) [1]

Problem 16: Print LCS

Algorithm:

```
def printLCS(S1, S2):
    m, n = len(S1), len(S2)
    dp = [[^0] * (n + 1) for _ in range(m + 1)]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if S1[i-1] == S2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Backtrack to construct LCS
    lcs = []
    i, j = m, n
    while i > 0 and j > 0:
        if S1[i-1] == S2[j-1]:
            lcs.append(S1[i-1])
            i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            i -= 1
        else:
            j -= 1

    return ''.join(reversed(lcs))
```

Backtracking Logic:

- If characters match, they're in the LCS: add and move diagonally
- Otherwise, move in the direction of the larger value [1]

Complexity:

- Time: $O(mn)$
- Space: $O(mn)$ [1]

Example: $S_1 = \text{”axyb”}$, $S_2 = \text{”abyxb”}$
Result: "axb" or "ayb" (both valid) [1]

Problem 17: LIS in O(n log n)

Algorithm: Use binary search with patience sorting approach.[1]

Key Idea: Maintain an array $\text{tails}[i]$ = smallest tail element of all increasing subsequences of length $i+1$.[1]

Algorithm:

```
import bisect

def LIS_fast(arr):
    tails = []
    parent = [-1] * len(arr)
    indices = []  # Track indices in tails

    for i, num in enumerate(arr):
        pos = bisect.bisect_left(tails, num)

        if pos == len(tails):
            tails.append(num)
            indices.append(i)
        else:
            tails[pos] = num
            indices[pos] = i

        if pos > 0:
            parent[i] = indices[pos - 1]

    # Reconstruct LIS
    lis_length = len(tails)
    return lis_length
```

Explanation:

- For each element, use binary search to find its position in tails
- If it extends the longest subsequence, append it
- Otherwise, replace the smallest element that's ≥ current element[1]

Correctness: The tails array is always sorted. Each element either extends the LIS or improves the potential for future extensions by providing a smaller tail.[1]

Complexity:

- Time: $O(n \log n)$ - n elements, each requiring $O(\log n)$ binary search
- Space: $O(n)$[1]

Problem 18: Longest Path in DAG

Problem: Find the longest weighted simple path from s to t in a DAG.[1]

State: $\mathrm{dp}[v]$ = length of longest path from s to vertex v.[1]

Base Case: $\mathrm{dp}[s] = 0$, $\mathrm{dp}[v] = -\infty$ for all $v \neq s$.[1]

Transition: Process vertices in topological order:
$\mathrm{dp}[v] = \max_{(u,v)\in E}\{\mathrm{dp}[u] + w(u,v)\}$

Algorithm:

```
def longestPath(G, s, t):
    # Step 1: Topological sort
    topo_order = topologicalSort(G)

    # Step 2: Initialize DP
    dp = {v: float('-inf') for v in G.vertices}
    dp[s] = 0

    # Step 3: Process in topological order
    for u in topo_order:
        if dp[u] != float('-inf'):
            for v, weight in G.adj[u]:
                dp[v] = max(dp[v], dp[u] + weight)

    return dp[t]
```

Subproblem Graph: The subproblem graph is the same as the input DAG G. Each vertex in G corresponds to a subproblem.[1]

Correctness: Since we process in topological order, when computing $\mathrm{dp}[v]$, all predecessors have been processed.[1]

Complexity:

- Time: $O(V + E)$ for topological sort + $O(V + E)$ for DP = $O(V + E)$
- Space: $O(V)$[1]

Problem 19: Longest Palindromic Subsequence

Problem: Find the longest palindrome that is a subsequence of input string.[1]

Key Insight: A string's longest palindromic subsequence = LCS of the string with its reverse.[1]

Algorithm:

```
def longestPalindromicSubsequence(s):
    reversed_s = s[::-1]
```

```
    return LCS_Length(s, reversed_s)
```

Why this works: A palindrome reads the same forwards and backwards. If a subsequence appears in both s and $\text{reverse}(s)$, it must be palindromic.[1]

Example: Input = "character"

- Reverse = "retcarahc"
- LCS = "carac"
- Length = 5[1]

Complexity:

- Time: $O(n^2)$ (LCS algorithm)
- Space: $O(n^2)$[1]

Problem 20: Bitonic TSP Tour

Problem: Find shortest bitonic tour (go rightward to rightmost point, then leftward back) through n points in $O(n^2)$ time.[1]

Assumptions: Points sorted by x-coordinate, no two points have same x-coordinate.[1]

State: $\text{dp}[i][j]$ (where $i > j$) = minimum length of bitonic path where:

- Rightward path ends at point i
- Leftward path ends at point j
- All points from 1 to i have been visited[1]

Base Case: $\text{dp} = 0$ (start at leftmost point).[2] [1]

Transition: For $i > j$:

$$\text{dp}[i][j] = \min \begin{cases} \text{dp}[i-1][j] + \text{dist}(i-1, i) & (\text{extend right path}) \\ \text{dp}[i-1][k] + \text{dist}(k, i) & \text{for all } k < j \ (\text{jump from left path}) \end{cases}$$

Algorithm:

```
def bitonicTSP(points):
    n = len(points)
    points.sort()  # Sort by x-coordinate

    dp = [[float('inf')] * n for _ in range(n)]
    dp[^1][^0] = 0

    for i in range(2, n):
        for j in range(i):
            # Extend right path from i-1 to i
            dp[i][j] = dp[i-1][j] + dist(points[i-1], points[i])
```

```
            # Jump to i from left path at j
            for k in range(j):
                dp[i][j] = min(dp[i][j],
                               dp[i-1][k] + dist(points[k], points[i]))

    # Close the tour
    result = float('inf')
    for j in range(n-1):
        result = min(result, dp[n-1][j] + dist(points[j], points[n-1]))

    return result
```

Complexity:

- Time: $O(n^2)$ - two nested loops with constant work
- Space: $O(n^2)$ [1]

Problem 21: Making Change with Coin Constraint

Part 1: Each coin used at most once

State: $\mathrm{dp}[i][v]$ = True if we can make value v using first i coins, each used at most once. [1]

Algorithm:

```
def canMakeChange(coins, v):
    n = len(coins)
    dp = [[False] * (v + 1) for _ in range(n + 1)]

    # Base case
    for i in range(n + 1):
        dp[i][^0] = True

    # Transition
    for i in range(1, n + 1):
        for value in range(v + 1):
            # Don't use coin i
            dp[i][value] = dp[i-1][value]

            # Use coin i (if possible)
            if value >= coins[i-1]:
                dp[i][value] |= dp[i-1][value - coins[i-1]]

    return dp[n][v]
```

Complexity:

- Time: $O(nv)$
- Space: $O(nv)$ [1]

Part 2: Greedy optimal for powers of c

Claim: When coins are $\{c^0, c^1, \ldots, c^k\}$, greedy algorithm is optimal.[1]

Proof by Exchange Argument:
Suppose optimal solution uses a_i coins of denomination c^i. If for some i, $a_i \geq c$, we can replace c coins of c^i with one coin of c^{i+1}, reducing total coins. This contradicts optimality.[1]

Therefore, in optimal solution, each denomination is used < c times. The greedy algorithm produces exactly this: use as many largest coins as possible (< c of each), which is optimal.[1]

Problem 22: Probability of k Heads with Biased Coins

Problem: Given n biased coins with probabilities $p_1, \ldots, p_n$, find probability of exactly k heads.[1]

State: $\mathrm{dp}[i][j]$ = probability of getting exactly j heads from first i coins.[1]

Base Cases:

- $\mathrm{dp} = 1$ (0 coins, 0 heads: probability 1)
- $\mathrm{dp}[j] = 0$ for $j > 0$[1]

Transition:
$$\mathrm{dp}[i][j] = \mathrm{dp}[i-1][j] \cdot (1 - p_i) + \mathrm{dp}[i-1][j-1] \cdot p_i$$

Interpretation: Either coin i is tails (probability $1 - p_i$) or heads (probability p_i).[1]

Algorithm:

```
def exactKHeads(probabilities, k):
    n = len(probabilities)
    dp = [[0.0] * (k + 2) for _ in range(n + 1)]
    dp[^0][^0] = 1.0

    for i in range(1, n + 1):
        for j in range(min(i, k) + 1):
            # Coin i is tails
            dp[i][j] = dp[i-1][j] * (1 - probabilities[i-1])

            # Coin i is heads
            if j > 0:
                dp[i][j] += dp[i-1][j-1] * probabilities[i-1]

    return dp[n][k]
```

Complexity:

- Time: $O(nk)$ (can be optimized to $O(n \cdot \min(k, n-k))$)
- Space: $O(nk)$ (can be optimized to $O(k)$)[1]

Problem 23: Change with At Most k Coins

Problem: Can we make value v using at most k coins from denominations $\{x_1, \ldots, x_n\}$?[1]

State: $\mathrm{dp}[i][c]$ = minimum number of coins to make value i (or ∞ if impossible), where c represents the count constraint.[1]

Better State: $\mathrm{dp}[i]$ = minimum coins for value i.[1]

Algorithm:

```
def canMakeChangeWithKCoins(denominations, v, k):
    dp = [float('inf')] * (v + 1)
    dp[^0] = 0

    for value in range(1, v + 1):
        for coin in denominations:
            if value >= coin and dp[value - coin] != float('inf'):
                dp[value] = min(dp[value], dp[value - coin] + 1)

    return dp[v] <= k
```

Complexity:

- Time: $O(nv)$
- Space: $O(v)$[1]

Problem 24: Match Winning Probability

Problem: Teams A and B play until one wins n games. A has won i, B has won j. Find probability A wins the match.[1]

State: $\mathrm{dp}[a][b]$ = probability that A wins when A needs a more wins and B needs b more wins.[1]

Base Cases:

- $\mathrm{dp}[b] = 1$ for all $b > 0$ (A already won)
- $\mathrm{dp}[a] = 0$ for all $a > 0$ (B already won)[1]

Transition: Each team has 50% chance of winning each game:
$\mathrm{dp}[a][b] = 0.5 \cdot \mathrm{dp}[a-1][b] + 0.5 \cdot \mathrm{dp}[a][b-1]$

Algorithm:

```
def matchWinProbability(i, j, n):
    need_A = n - i
    need_B = n - j

    dp = [[0.0] * (need_B + 1) for _ in range(need_A + 1)]

    # Base cases
```

```
    for b in range(need_B + 1):
        dp[^0][b] = 1.0
    for a in range(1, need_A + 1):
        dp[a][^0] = 0.0

    # Fill table
    for a in range(1, need_A + 1):
        for b in range(1, need_B + 1):
            dp[a][b] = 0.5 * dp[a-1][b] + 0.5 * dp[a][b-1]

    return dp[need_A][need_B]
```

Example: $n = 5$, $i = 4$, $j = 2$

- A needs 1 win, B needs 3 wins
- Probability A wins = 7/8 [1]

Complexity:

- Time: $O((n-i)(n-j))$
- Space: $O((n-i)(n-j))$ [1]

Problem 25: Local Sequence Alignment

Problem: Find substrings x' of x and y' of y with highest-scoring alignment. [1]

State: $\text{dp}[i][j]$ = best alignment score for substrings ending at $x[i]$ and $y[j]$. [1]

Base Cases: $\text{dp}[i] = 0$, $\text{dp}[j] = 0$ (empty substrings have score 0). [1]

Transition: (Smith-Waterman algorithm)

$$\text{dp}[i][j] = \max \begin{cases} 0 & (\text{start new alignment}) \\ \text{dp}[i-1][j-1] + \delta(x[i], y[j]) & (\text{match/mismatch}) \\ \text{dp}[i-1][j] + \delta(x[i], \text{gap}) & (\text{gap in } y) \\ \text{dp}[i][j-1] + \delta(\text{gap}, y[j]) & (\text{gap in } x) \end{cases}$$

Algorithm:

```
def localAlignment(x, y, delta):
    m, n = len(x), len(y)
    dp = [[^0] * (n + 1) for _ in range(m + 1)]
    max_score = 0
    max_pos = (0, 0)

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            match = dp[i-1][j-1] + delta[x[i-1]][y[j-1]]
            delete = dp[i-1][j] + delta[x[i-1]]['gap']
            insert = dp[i][j-1] + delta['gap'][y[j-1]]

            dp[i][j] = max(0, match, delete, insert)
```

```
            if dp[i][j] > max_score:
                max_score = dp[i][j]
                max_pos = (i, j)

    # Backtrack from max_pos to reconstruct alignment
    return max_score, backtrack(dp, x, y, max_pos, delta)
```

Key Difference from Global Alignment: We can start alignment anywhere (score can reset to 0).[1]

Complexity:

- Time: $O(mn)$
- Space: $O(mn)$[1]

Problem 26: GCD of Consecutive Fibonacci Numbers

Problem (Not DP): Prove $\gcd(F_n, F_{n-1}) = 1$.[1]

Proof by Strong Induction:

Base Cases:

- $\gcd(F_1, F_0) = \gcd(1, 1) = 1$ ✓
- $\gcd(F_2, F_1) = \gcd(1, 1) = 1$ ✓[1]

Inductive Step: Assume $\gcd(F_k, F_{k-1}) = 1$ for all $k < n$. We'll prove for n:

By Euclidean algorithm property: $\gcd(F_n, F_{n-1}) = \gcd(F_{n-1}, F_n \mod F_{n-1})$

Since $F_n = F_{n-1} + F_{n-2}$, we have:
$F_n \mod F_{n-1} = F_{n-2}$

Therefore:
$\gcd(F_n, F_{n-1}) = \gcd(F_{n-1}, F_{n-2})$

By inductive hypothesis, $\gcd(F_{n-1}, F_{n-2}) = 1$

Thus, $\gcd(F_n, F_{n-1}) = 1$ ✓[1]

This completes all 26 problems from your Dynamic Programming problem set! Each solution includes clear state definitions, transitions, algorithms, correctness arguments, and complexity analysis as requested.[1]

⁂

1. AAD_QB_1_3-DP.pdf
2. AAD_QB_1_5-All-Topics.pdf